

Operating Systems with Linux

Module 1 – Exam Study Notes

High-yield notes, comparisons, diagrams, and question bank

Exam Focus

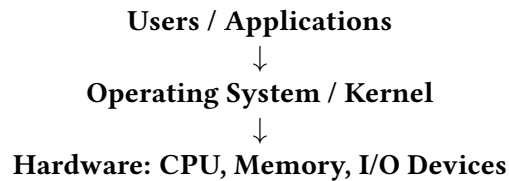
Exam strategy. Module 1 contributes only two questions in the stated paper pattern, with a maximum of one 10-mark question. Therefore, the goal is not to memorize every sentence. Prioritize definitions, mechanisms, comparisons, diagrams, and answer structures that can be expanded into 5–10 marks.

Contents

1. Operating System Fundamentals

1.1 What is an Operating System?

An Operating System (OS) is the software layer that acts as an interface between users/applications and computer hardware. The kernel is the part of the OS that runs continuously and manages hardware and system resources.



1.2 Main Objectives of an OS

1. **Convenience** – make the computer system easier to use.
2. **Efficiency** – use system resources efficiently.
3. **Evolvability** – support testing, development, and addition of new components.

1.3 OS as an Extended Machine

The OS hides complicated hardware details and presents simpler abstractions to users and applications.

Example: disk access

- Without an OS, low-level disk handling can involve block addresses, sector calculations, and physical layout details.
- With an OS, a user/application can work with a logical file name and location instead of directly manipulating the physical disk.

Remember

Think: **Extended Machine = Abstraction**. The OS hides hardware complexity and gives the user a convenient, higher-level interface.

1.4 OS as a Resource Manager

The OS manages competing demands for CPU, memory, storage, and I/O resources.

- Keeps track of allocated resources.
- Grants resource requests when resources are available.
- Accounts for resource usage.
- Resolves conflicts between programs/users.

1.5 Resource Multiplexing

Type	Meaning	Typical example
Time multiplexing	A resource is shared turn-by-turn over time.	CPU scheduling: P1, then P2, then P3.
Space multiplexing	A resource is divided into portions that can be assigned simultaneously.	Main memory or disk storage divided among users/processes.

1.6 OS from Two Viewpoints

User viewpoint: depends on environment. Stand-alone systems emphasize ease of use; networked systems emphasize resource utilization; handheld systems emphasize individual usability; embedded systems emphasize reliable operation with little/no user intervention.

System viewpoint: the OS has two primary roles:

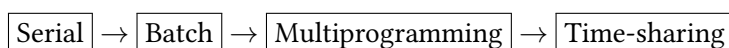
1. **Resource allocator** – manages CPU time, memory and I/O resources and aims for efficient/fair distribution.
2. **Control program** – manages user programs, controls I/O, prevents errors and improper system use.

Exam Focus

Likely 5-mark question: Explain the objectives of an operating system and describe the OS as an extended machine and resource manager.

2. Evolution of Operating Systems

2.1 The Evolution Chain



Remember

The evolution is driven by problems: **wasted machine time** → **automation** → **CPU idle during I/O** → **need for interactive multi-user computing**.

2.2 Serial Processing

Period: late 1940s–mid 1950s.

- No operating systems existed.
- Programmers interacted directly with hardware.
- Programs were written in machine language.
- Input used punched cards; output used line printers.
- Users reserved machine time and accessed the computer sequentially.
- Output could take minutes, hours, or days.

Main problem: very poor utilization. If a programmer finished early, the reserved machine time could remain unused.

System software improvements

Later tools included language translators, loaders, linkers, debuggers, library routines, and I/O routines. They made serial processing more efficient but did not solve the fundamental resource-utilization problem.

2.3 Batch Processing

Core idea: group similar jobs into batches and execute them automatically one after another.

1. Users submit jobs to an operator.
2. Similar jobs are grouped into a batch.
3. The OS loads and executes jobs sequentially.
4. After one job finishes, the next begins automatically.

Advantages:

- Reduced manual intervention.
- Better utilization than serial processing.
- Suitable for repetitive and large data-processing tasks.

Limitations:

- No interaction while a job executes.
- Long turnaround time.

- Debugging is difficult.

2.4 SPOOLing

SPOOL stands for **Simultaneous Peripheral Operation Online**. It addresses the speed mismatch between a fast CPU and slower I/O devices.

Input/cards → tape/disk → CPU → tape/disk → printer/output

Idea: jobs are copied to faster intermediate storage, allowing the CPU to work without waiting directly for slow peripherals. Later, output can be sent from storage to the printer.

2.5 Multiprogramming

Core idea: keep multiple jobs in main memory simultaneously. When one job waits for I/O, the CPU executes another ready job.

Main Memory

OS Kernel

Job P1

Job P2

Job P3

Free Memory

Example:

P1: CPU → I/O wait → CPU

P2: CPU → I/O wait → CPU

When P1 performs I/O, P2 can use the CPU. The objective is to keep the CPU busy whenever a ready job exists.

Important terms:

- **Degree of multiprogramming:** number of jobs competing for resources / residing for execution in the system.
- **Job scheduling:** OS decides which job should be loaded into memory.
- **CPU scheduling:** OS chooses which ready job receives the CPU.
- **Memory management:** OS manages memory partitions/space for multiple jobs.

2.6 Time-sharing

Time-sharing is an extension of multiprogramming in which multiple users interact with the system simultaneously. CPU time is divided into small time slots/time slices, and the CPU switches rapidly between users/processes.

Time → P1 – P2 – P3 – P1 – P2 – P3 – ...

Result: each user gets the impression of having a dedicated computer while the system is actually sharing resources.

Advantages: fast response, multiple users, interactive computing, efficient CPU use, easier development/debugging.

Limitations: greater complexity, security/data-protection challenges, possible performance degradation with many users, and need for sophisticated scheduling.

2.7 Evolution Comparison

	Serial	Batch	Multiprogramming	Time-sharing
Users/jobs	Single/sequential users	Multiple jobs	Multiple jobs	Multiple users/processes
Execution	One at a time	Sequential batch jobs	Jobs interleaved around I/O waits	Rapid CPU switching/time slices
Interaction	None/very limited	None during execution	Generally non-interactive	Interactive
Main goal	Basic execution	Automation	CPU/resource utilization	Fast interactive response
CPU utilization	Low	Better than serial	High	Very high

3. Types of Operating Systems

3.1 Batch Operating System

Processes groups of jobs without user interaction during execution. Similar jobs are collected and executed automatically one after another.

3.2 Time-Sharing Operating System

Allows multiple users/processes to share CPU time using small time slices. Rapid switching creates interactive, simultaneous use.

3.3 Real-Time Operating System (RTOS)

An RTOS is designed to provide responses within strict time constraints. Correctness depends not only on the result but also on when the result is produced.

How it works: tasks are assigned priorities; critical tasks receive immediate CPU access; predictable response times are a key requirement.

Hard real-time Missing a deadline is unacceptable. Examples in the study material include aircraft control, medical life-support, and nuclear power plant controls.

Soft real-time Deadlines are important, but occasional misses are acceptable. Examples include video streaming, online gaming, and multimedia applications.

Feature	Hard RTOS	Soft RTOS
Deadline	Strict and mandatory	Flexible/desirable; occasional misses tolerated
Miss consequence	Safety risk/catastrophic consequence possible	Reduced performance/quality; system continues
Response Scheduling	Guaranteed and predictable Deterministic	Fast but not always guaranteed Best-effort
Application	Safety/mission-critical	Multimedia/commercial

3.4 Multiprogramming OS

Allows multiple programs to reside in main memory simultaneously and share the CPU. When one program waits for I/O, another ready program gets the CPU.

Do Not Confuse

Multiprogramming is not the same as parallel execution. The core idea in the material is concurrent/interleaved execution: the CPU switches among jobs so that it is not idle during I/O waits.

3.5 Batch vs Multiprogramming

Feature	Batch OS	Multiprogramming OS
Jobs in memory	One job at a time in the basic model	Multiple jobs simultaneously
CPU during I/O	May remain idle	Executes another ready job
Utilization	Lower	Higher
Memory management	Simpler	More sophisticated
Scheduling	Sequential/FCFS in the material	More sophisticated scheduling needed
Concurrency control	Simpler	More protection/concurrency control needed

4. Operating System Components and Services

4.1 Process Management

A **process** is a program under execution/an executing set of machine instructions. A program is passive; a process is active.

OS process-management responsibilities include:

- create and delete user/system processes;
- suspend and resume processes;
- provide mechanisms for process communication;
- provide mechanisms for dealing with deadlock.

4.2 Memory Management

Main memory holds instructions/data currently needed by the CPU and is directly accessible to the CPU. In multiprogramming, several programs reside in memory simultaneously.

Responsibilities:

- allocate and deallocate memory;
- decide which ready process receives available memory;
- track which portions of memory are allocated and to which processes.

4.3 Disk Management

Secondary storage provides persistent storage because main-memory contents are lost when power is removed.

- allocate disk space;
- manage free disk space;
- perform disk scheduling.

4.4 File Management

The OS provides a logical storage abstraction called a **file** and hides physical storage details.

- create/delete files and directories;
- map files onto secondary storage;
- back up files to stable storage;
- provide operations for manipulating file/directory contents.

Common file operations include create, delete, read, write, open, seek, rename, append, and close.

4.5 I/O Management

The OS controls I/O devices and provides a device-independent interface. Device-driver modules hide device-specific details and present a uniform access interface.

4.6 Protection and Security

Protection controls which processes/programs/users may access system resources.

Security protects the system against unauthorized access and attacks. Authentication such as username/password is one security mechanism.

4.7 Other OS Services

1. User interface (CLI, batch interface, GUI)
2. Program execution
3. I/O operations
4. File-system manipulation
5. Communication
6. Error detection
7. Resource allocation
8. Protection and security
9. Accounting

Exam Focus

Likely 10-mark question: Explain the major components/services of an operating system. For each, give its purpose and major responsibilities. A strong answer should cover process, memory, disk, file, I/O, protection/security, and OS services.

5. Operating System Structures

Different OS designs organize components differently. The purpose of structure is to make the large OS manageable by dividing it into components with defined functions and interfaces.

5.1 Monolithic Architecture

In a monolithic design, major OS components operate closely together, traditionally in a single large kernel/address space.

Advantages:

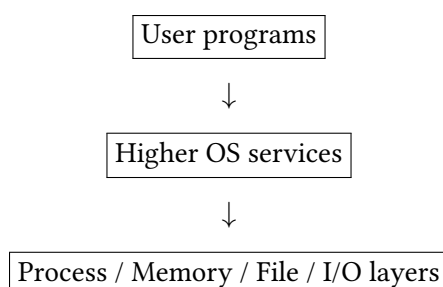
- High performance because components communicate directly within the kernel.
- Provides functions such as CPU scheduling, memory management, and file management through system calls.
- Simple single-binary organization in the model presented in the material.

Disadvantages:

- Components are interdependent.
- Failure of a component can affect the whole system.
- Adding functionality can require changes to the OS.

5.2 Layered Structure

The OS is divided into layers. Each layer uses services of lower layers and provides services to higher layers.





Hardware

Core idea: modular organization through clear layer boundaries.

Exam point: mention easier organization/debugging and the overhead/rigidity that can result from strict layer boundaries if asked for advantages/disadvantages.

5.3 Microkernel

A microkernel keeps only essential mechanisms in the kernel and moves many services into user-space processes. The approach aims for modularity, isolation, and easier extension.

Exam comparison:

Feature	Monolithic	Microkernel
Kernel size	Large	Small/minimal
Services	Many OS services in kernel space	Many services moved to user space
Isolation	Lower between kernel components	Better isolation between service processes
Performance	Generally faster direct communication	IPC overhead may reduce performance
Modularity	Lower	Higher

6. Linux as the Case Study

6.1 What is Linux?

The study material describes Linux as a UNIX-like operating system built around the Linux kernel. The kernel manages hardware and resources; a complete Linux distribution combines the kernel with supporting software, libraries, and utilities.

6.2 Linux History

The material states that Linus Torvalds designed the free and open-source Linux kernel in 1991, inspired by UNIX and MINIX. The kernel subsequently became the basis for complete Linux distributions supported by open-source software.

6.3 Linux Distribution

A Linux distribution is a complete Linux-based operating system consisting of the Linux kernel plus supporting libraries and software. Examples listed in the material include Ubuntu, Debian, Fedora, Linux Mint, Manjaro, openSUSE, and others.

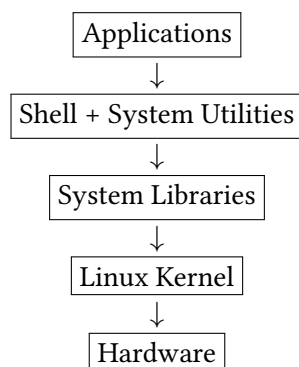
Remember

Linux kernel *eq* complete distribution.

Kernel = core that manages hardware/resources.

Distribution = kernel + libraries + utilities + other software needed for a usable system.

6.4 Linux Architecture / Layers



Hardware: CPU, RAM, storage, and I/O devices.

Kernel: core; manages memory, resources, devices, processes, application interaction, and security.

System libraries: reusable code that provides standardized interfaces for applications to interact with system functionality.

Shell: user interface that interprets commands and forwards requests toward the kernel.

System utilities: programs for administration, configuration, software installation, networking, monitoring, users, permissions, and related tasks.

6.5 Linux Usage

The material highlights servers/hosting, development, desktop use, cybersecurity, embedded systems, supercomputers, and education.

7. Linux Shell and Processes

7.1 Shell

The shell is a user interface for Linux. It accepts commands, interprets them, and executes the requested operations. Examples of common shell commands in the study material include:

`ls` – list directory contents; `cd` – change directory; `pwd` – print current path.

`cp` – copy; `mv` – move; `rm` – remove.

`free` – memory usage; `df` – disk usage; `top` – process monitoring.

7.2 Process in Linux

A Linux process is a running instance of a program. Each process has a unique Process ID (PID).

Foreground process: interactive, normally receives keyboard input and sends output to the screen.

Background process: runs without requiring direct user input and allows other processes to continue.

7.3 Basic Process Commands

Command	Purpose
<code>ps -e</code>	View running processes.
<code>ps -ef</code>	View detailed process information.
<code>top</code>	Dynamic real-time process view.
<code>sleep 100 &</code>	Start a command in the background.
<code>kill PID</code>	Send a termination signal to a process.
<code>kill -9 PID</code>	Forcefully terminate a process using SIGKILL.
<code>nice -n 5 sleep 100</code>	Start a process with a specified nice value.
<code>renice 10 PID</code>	Change the nice value/priority of an existing process.

7.4 Special Process Types

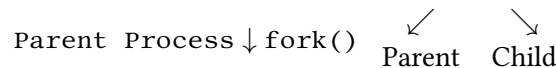
Daemon: background process that starts at system startup and runs continuously, often providing services to other processes.

Zombie: process that has completed execution but still has an entry in the process table.

Orphan: child process whose parent has terminated; the material states that the init process adopts it.

8. Parent and Child Processes

A process can create another process, forming a parent-child relationship. In the material, this is achieved using the `fork()` system/library call.



8.1 `fork()` Return Values

On successful `fork()`:

- Parent receives the child's PID.
- Child receives return value **0**.

On failure:

- Parent receives **-1**.
- No child is created.

8.2 Parent vs Child

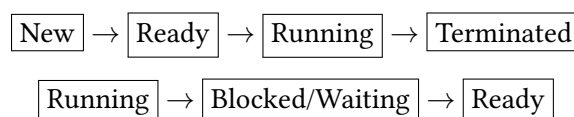
- A parent can have multiple child processes.
- A child has one parent process.
- A child is initially created as a copy of the parent and inherits most attributes.
- Parent-child processes allow concurrent execution of related tasks.

Exam Focus

Very likely short/long question: Explain parent and child processes in Linux. Describe `fork()` and its return values on success and failure.

9. Process States and PCB

9.1 Five Basic Process States



New: process is being created.

Ready: process is prepared to execute and waiting for CPU allocation.

Running: instructions are currently being executed by the CPU.

Blocked: process cannot continue until the required event/I/O completes.

Terminated: process has finished execution.

9.2 Process Control Block (PCB)

A PCB stores important information required to manage a process. The material lists:

1. Process state
2. Process number/PID

3. Program counter
4. Files and registers information
5. CPU information
6. Memory information

Remember

PCB = the OS's record of a process. If asked "What is a PCB?", define it first, then list its contents, then explain that it enables the OS to manage and resume the process.

10. Linux File Hierarchy Structure

The Filesystem Hierarchy Standard (FHS) defines directory structure and directory contents in Unix-like systems. In the material, everything is organized under the root directory /.

Directory	Exam meaning
/	Root of the filesystem; all other directories branch from it. /root is the root user's home and is not the same as /.
/bin	Essential user commands/binary executables.
/boot	Files required for booting, including kernel/initrd and bootloader-related files such as GRUB configuration.
/dev	Device files; interfaces representing devices such as disks and terminals.
/etc	Host-specific/system-wide configuration files.
/home	Home directories for regular users and their personal files/settings.
/lib	Shared/dynamic libraries required by applications/system programs.
/media	Mount locations commonly used for removable media.
/mnt	Temporary mount point for filesystems/external drives.
/opt	Add-on/third-party software and packages.
/proc	Process/kernel information exposed as a virtual filesystem.
/sbin	Essential system-administration binaries.
/srv	Service-specific data, such as data served by servers.
/tmp	Temporary files created by users/programs.
/usr	Secondary hierarchy containing many user utilities, applications, libraries and documentation.
/var	Variable data such as logs, caches, spools and other data that changes during system operation.

Remember

High-return directories to memorize first: /, **/bin**, **/boot**, **/dev**, **/etc**, **/home**, **/lib**, **/mnt**, **/opt**, **/proc**, **/sbin**, **/tmp**, **/usr**, **/var**. If time is short, learn the one-line purpose of each rather than memorizing examples.

11. High-Yield Comparison Sheet

11.1 Program vs Process

	Program	Process
Nature	Passive entity	Active entity
Meaning	Set of instructions/code	Program currently executing
State	No execution state by itself	Has a process state
Resources	Does not represent allocated execution resources	Uses CPU, memory, files, I/O etc.
Identity	No PID simply because it is code	Has a PID in Linux

11.2 Protection vs Security

	Protection	Security
Focus	Controlled access to resources	Protection against unauthorized access/attacks
Question	Who may access this resource?	How do we keep the system safe from unauthorized users/attacks?
Example	Process allowed/denied access to memory/file	Authentication and defense against attacks

11.3 Kernel vs Shell vs System Library vs Utility

Component	Role
Kernel	Core; manages CPU/processes, memory, devices, resources and security.
System library	Reusable code/interface used by applications to access system functionality.
Shell	Command-oriented user interface that interprets commands and requests operations.
System utility	Administrative/support programs for installation, networking, monitoring, users, permissions, etc.

11.4 Time Multiplexing vs Space Multiplexing

	Time	Space
Sharing method	Turn-by-turn	Partition-by-partition
Example	CPU	RAM/disk

12. 10-Mark Answer Templates

Question A: Explain the evolution of operating systems.

[10 marks]

Answer Skeleton

Write in this order:

1. Introduction: need for improving resource utilization and user productivity.
2. Serial processing – definition, characteristics, problems.
3. Batch processing – definition, operation, advantages/limitations.
4. SPOOLing – CPU/I/O speed mismatch and solution.
5. Multiprogramming – multiple jobs in memory; switch to another job during I/O.
6. Time-sharing – extension of multiprogramming; time slices and interactive users.
7. Draw the evolution chain and finish with a comparison table.

Question B: Explain OS components/services.**[10 marks]****Answer Skeleton**

Start with a definition of OS, then cover:

1. Process management
2. Memory management
3. Disk management
4. File management
5. I/O management
6. Protection and security
7. User interface
8. Program execution
9. Communication

10. Error detection/resource allocation/accounting

Give one or two responsibilities under each heading. Use a diagram showing OS between applications and hardware.

Question C: Explain Linux architecture and process management.**[10 marks]****Answer Skeleton**

Possible structure:

1. Define Linux and Linux distribution.
2. Draw: Applications → Shell/Utilities/Libraries → Kernel → Hardware.
3. Explain kernel, system libraries, shell and utilities.
4. Define process and PID.
5. Explain foreground/background processes.
6. Explain `fork()`, parent/child and return values.
7. List process states and PCB contents.
8. Mention commands such as `ps`, `top`, `kill` as Linux examples.

13. Question Bank**Definitions / 2–3 marks**

1. Define an operating system.
2. What are the three objectives of an OS?
3. What is meant by OS as an extended machine?
4. What is resource multiplexing?
5. Distinguish time multiplexing and space multiplexing.
6. Define batch operating system.
7. What is SPOOLing?
8. Define multiprogramming.
9. Define time-sharing.
10. What is an RTOS?
11. Differentiate hard and soft real-time systems.
12. Define a process.
13. What is a PID?
14. What is a PCB?
15. What is `fork()`?
16. What are daemon, zombie and orphan processes?
17. What is a Linux distribution?
18. What is the role of the Linux kernel?

19. What is the shell?
20. What is the purpose of `/etc`, `/home`, `/boot`, and `/proc`?

5-mark questions

1. Explain the OS as an extended machine and resource manager.
2. Explain the objectives and primary roles of an OS.
3. Explain serial, batch, multiprogramming and time-sharing systems.
4. Explain SPOOLing and the problem it solves.
5. Compare batch and multiprogramming operating systems.
6. Explain hard real-time and soft real-time systems.
7. Explain major OS system components.
8. Explain OS services.
9. Explain Linux architecture/layers.
10. Explain Linux processes and basic process-management commands.
11. Explain parent and child processes using `fork()`.
12. Explain the five basic process states and PCB.
13. Explain the Linux file hierarchy with important directories.

10-mark questions

1. Explain the evolution of operating systems from serial processing to time-sharing, including SPOOLing and multiprogramming.
2. Explain the components and services of an operating system.
3. Explain different operating-system structures and compare their characteristics.
4. Explain Linux architecture and describe the role of kernel, system libraries, shell and system utilities.
5. Explain Linux process management, including process types, process states, PCB, parent/child processes and `fork()`.
6. Explain the Linux File Hierarchy Structure and the purpose of important directories.

14. Last-Minute Revision Page

OS Interface + resource manager + control program.	mandatory; Soft = occasional misses tolerated.
Objectives Convenience + Efficiency + Evolvability.	Process Program under execution; PID identifies it.
Multiplexing Time = turn-by-turn. Space = portions.	fork() Parent gets child PID; child gets 0; failure gives -1.
Evolution Serial → Batch → Multiprogramming → Time-sharing.	States New, Ready, Running, Blocked, Terminated.
Batch Group jobs; sequential execution; no interaction.	PCB State, PID, PC, files/registers, CPU info, memory info.
SPOOL Use intermediate storage to reduce CPU waiting for slow I/O.	Linux stack Applications → Shell/Libraries/Utilities → Kernel → Hardware.
Multiprogramming Several jobs in memory; another runs during I/O wait.	Filesystem Everything starts at <code>/</code> ; know <code>/bin</code> , <code>/boot</code> , <code>/dev</code> , <code>/etc</code> , <code>/home</code> , <code>/lib</code> , <code>/mnt</code> , <code>/opt</code> , <code>/proc</code> , <code>/sbin</code> , <code>/tmp</code> , <code>/usr</code> , <code>/var</code> .
Time-sharing Time slices + rapid switching + interactive users.	Core commands <code>ls</code> , <code>cd</code> , <code>pwd</code> , <code>cp</code> , <code>mv</code> , <code>rm</code> , <code>ps</code> , <code>top</code> , <code>kill</code> , <code>free</code> , <code>df</code> .
RTOS Correctness includes timing. Hard = deadline	

Source Scope

These notes are consolidated from the seven Module 1 study files supplied for this session: Introduction to Operating Systems; Evolution of OS; Types of Operating Systems; Operating System Structures; Linux History/Linux Operating System Layers; Linux Shell and Processes/Parent–Child Processes; and Linux File Hierarchy Structure. Terminology and emphasis follow those materials.

Study principle: understand the mechanism first, then memorize the exam wording.